Menu unlocks and medal limits – Pilotwings 64 (USA)

Research for stage 6 (Classes tab: medal limits per vehicle and class; Menu tab: unlock conditions per menu field), 22 October 2026. Section 6: what Step 37 built. Function names from the decompilation (https://github.com/gcsmith/Pilotwings64Decomp, MIT license). ROM offset = RAM − 0x80278AD0. Verified in the disassembly unless marked "(C only)".

1. Menu fields

Level select (game state VEHICLE_CLASS_SELECT, func_8030D930, ROM 0x94E60) has 25 fields:

Page	Fields	Save data
1 (licenses)	3 vehicles × 4 classes (cell = class × 3 + vehicle)	veh 0–2, cls 0–3
2 (bonus)	3 games × 3 levels (cell = game × 3 + level)	veh 3–5, cls = level
2 (Birdman row)	4 stages	veh 6, cls = stage

Cell textures: 0x14C locked (also the overlay), 0x14D open without medal, 0x150 bronze, 0x14F silver, 0x14E gold. No "perfect" sprite. Birdman cells never show medals. The file menu (fileMenu_802E8AF0, ROM 0x70020) shows a medal grid (textures 0x148–0x14B) and locks of the bonus games and the Birdman icon (sFileMenu_803624E0[3], …E3); display only.

2. How the original decides what is open

All in level_select.c:

- **func_8030D9C8** (ROM 0x94EF8, page 1) and **func_8030DED0** (ROM 0x95400, page 2) compute the sprites and four arrays:
  - D_8036A8CC[3] (RAM 0x8036A8CC): highest open class per vehicle. The class loop stops at the first class without a medal (medal = total ⩾ gMed[class] bronze and every test played, func_8032BE8C). If any vehicle has 0, all vehicles are set back to 0 (Class A needs a Beginner medal with all three vehicles).
  - D_8036A8D8[4]: [c] = all three vehicles have silver in class c (all tests played). [1..3] open Cannonball, Sky Diving, Jumble Hopper. At the end [0] is reused as "some Birdman stage is open" (func_8030E7F4).
  - D_8036A8D4[4]: Birdman stage open. [0] = D_8036A8D8[0] (silver in Beginner, all three); [i] = D_8036A8D8[i] and all three levels of bonus game i ⩾ gMed[4].silver (no "all played" check).
  - D_8036A8D0[3]: highest open level per bonus game (level n+1 needs level n ⩾ gMed[4].bronze, no "all played"), −1 if the game is closed.
- **D_8034F7BC** (RAM 0x8034F7BC): leftover debug "all open" flag, only cleared (ROM 0x95994), partial effect.

- **Navigation** func_8030EA54 (ROM 0x95F84): the row is clamped to D_8036A8CC; the bonus page is entered from page 1 column 2 (Gyrocopter), row = game + 1 (also clamped by D_8036A8CC[2]), row 0 = Birdman. Bonus columns clamp to D_8036A8D0. Confirming a field has **no** unlock check (the cursor is trusted).
- The arrays assume prefix-contiguous classes and levels, so arbitrary unlock patterns need a new navigation.

Verified rules (save tab): Class A: all three Beginner medals; Class B/Pilot per vehicle: medal in the class before (and all earlier); bonus games: silver in Class A/B/Pilot with all three (all played); bonus levels 2/3: ⩾ 70 in the level before; Birdman stage 1: silver in Beginner with all three; stages 2–4: silver in Class A/B/Pilot with all three and ⩾ 80 in every level of Cannonball/ Sky Diving/Jumble Hopper.

3. Medal limits

gMedalPointRequirements: RAM 0x8034FBD4, ROM 0xD7104, 8 × {bronze, silver, gold} u16: {70,80,90}, {140,160,180}, {210,240,270} ×2, {70,80,90} ×3, {0,0,0} (Birdman, 280/320/360 with bonus-birdman-v1).

Every read (all `lui 0x8035` / −0x42C):

ROM	Function	Entry
0x70228	fileMenu_802E8AF0	[class] b/s/g
0x702F8	fileMenu_802E8AF0	[i+1].silver, [0].silver (0x70418)
0x70478	fileMenu_802E8AF0	[game+4].silver
0x95108	func_8030D9C8	[class] b/s/g
0x952BC	func_8030D9C8	[class].silver
0x95344	func_8030D9C8	[4].silver
0x95694	func_8030DED0	[class].silver
0x9571C	func_8030DED0	[4] b/s/g, [4].silver
0x971FC	func_8030F818	[class] or [4] bronze (points text shown)
0xB36EC	levelSetPointsToNextMedal (ROM 0xB36A4)	[arg]
0xB3970	func_8032C3C4 (ending, lowest medal)	[class], bonus [level+4]

levelSetPointsToNextMedal callers: 0xB311C/0xB3194/0xB31D0 scoring (class or veh+1), 0x93C64 stage screen (class or veh+1), 0xA9114 medal ceremony (class or 4; Birdman → module_ceremony_medal), 0xCE518/0xCE530 Cannonball total (fixed 4), 0xCE944 total screen (class or veh+1). The bonus entry is chosen five different ways (4, veh+1, level+4,

CLASS_COUNT, game+4); harmless only while entries 4–6 are equal. "Perfect" = 100 × tests (main), 100 (bonus).

levelGetTotalPoints (ROM 0xB3250) goes to the module since Step 36 (module_level_points). func_8032BE8C ("all tests played", ROM 0xB33BC) is called from the file menu and the two level-select functions only.

4. Per vehicle and class data

- Medal limits (above); class summary texts E/A/B/P_S3_* (shared by the three license vehicles), BONUS_S3_* (all bonus games and levels), BD_x_S3_* (Birdman, generated since Step 36).
- Island per vehicle and class: D_8034F420[7][4] at ROM 0xD6950 (test overview, globe, island name); the flown map comes from the mission.
- Tests per class: counted at boot from the missions (taskGetTestCount).
- Ending: credits_8030CC48 uses func_8032C27C (all perfect) and func_8032C3C4 (lowest medal).

5. Patch plan (proposed)

- Data area: medal limits [7 vehicles][4 classes][3] and the unlock conditions of the 25 fields.
- One module function computes the open mask; func_8030D9C8 and func_8030DED0 are replaced at their entry (`j`), filling the same sprites and arrays; func_8030EA54 (navigation) gets a module version for arbitrary patterns, with a check on confirm; the file menu's lock flags come from the same mask.
- All medal table reads above go to the new table.

6. Built in Step 37: Classes tab (medal limits and islands per class)

- **Settings:** `ClassSettings` per vehicle and class (level, stage) in project.json (`Classes`): own limits (bronze, silver, gold = points of the whole class) and/or an island. "Perfect" stays 100 per test (a bonus level 100, a Birdman stage with points 100 per mission with points). Validation: 0–9999, not falling from bronze to gold, gold ⩽ perfect. A Birdman stage with points and without own limits keeps 70/80/90 per mission with points (own limits are absolute numbers; decision of the user).
- **Class table** (data area 0x801FDC00, header +0x18 = its address, 0xC4 bytes): u16 limits[7][4][3], u8 flags[7][4] (1 own limits, 2 own medal texts). Only built when a class has own limits; entries without own limits hold the original values.
- **Module** (with a class table): entry `j` patches of func_8030D9C8 (0x94EF8), func_8030DED0 (0x95400), fileMenu_802E8AF0 (0x70020), levelSetPointsToNextMedal (0xB36A4) and func_8032C3C4 (0xB38F4); the bronze load of func_8030F818 (0x971F4–0x971FC) calls module_bronze_t2; with own texts the total screen's text call (0xCEA44) goes to module_tot_medal_text. The menu functions are rewritten in C with the same sprites and arrays and the original unlock rules, but with the limits of each vehicle and class and without the "all tests played" check (decision of the

user: only the sums count; with the original limits a medal needs every test anyway). levelSetPointsToNextMedal uses the player's vehicle and class (every caller asks about the class being played).

- **Texts:** a class with own limits gets own bronze/silver/gold texts (HG_A_S3_GOLD, SD_L2_S3_SILVER …), written from the project's shared text of the class (A_S3_GOLD, BONUS_S3_GOLD) with the new number and the header moved by 5 pixels per digit; needs the code fix expanded-texts-v1; locked in the Texts tab. Perfect/"great" stay shared. The Birdman's BD_x_S3_* texts take its own limits.
- **Island:** COMM byte 3 of every mission of the class (the island the game loads) and D_8034F420 (test screen).
- **Save tab:** counts with the project's limits and without the "all played" check.
- **Import:** class table and island table are read back; generated texts do not count as a changed text file.
- **Checked:** VM: the module's menu functions, the ending's lowest medal and "points to the next medal" give exactly the original results on 150/100/200 random save states (class table with the original limits); own limits change the medals and unlocking as expected; the total screen asks for HG_A_S3_GOLD, SD_L1_S3_SILVER, GC_E_S3_GOLD and keeps A_S3_GREAT. Emulator: file menu, level select (both pages) and pilot select with own limits and perfect.eep.